ASSIGNMENT 6 REPORT

Automation, Capacity Planning & Centralized Logging

MicroShop - Containerized Microservices System

Introduction to SRE | 2026

GitHub: https://github.com/Nurlybek-Nurzhan/Assignment_4-5_SRE

1. Overview

This report documents the work completed for Assignment 6 of the Introduction to SRE course. Building on the containerized MicroShop system established in Assignments 4 and 5, Assignment 6 introduces production-grade operational capabilities: automated alerting, centralised log aggregation, load-based capacity planning, and infrastructure-level security hardening.

The following capabilities were implemented and verified end-to-end:

- Prometheus alert rules (4 rules) wired to Alertmanager with severity-based routing
- Alertmanager configuration with critical (15 min) and warning (1 h) repeat intervals
- Loki + Promtail pipeline: Docker container logs collected and queryable in Grafana
- Grafana dashboard extended with p95/p50 latency panels and a live Loki logs panel
- Load simulation script (load_test.py) for concurrent stress testing
- AWS Security Group hardened - Alertmanager port 9093 restricted; ports 8001–8005 never opened
- Docker Compose security: microservice ports have no host binding; only Nginx port 80 is public
- Database credentials externalised to .env (not hardcoded in docker-compose.yml)

2. System Architecture

MicroShop consists of five Python/FastAPI microservices behind an Nginx reverse proxy, backed by PostgreSQL. The monitoring stack runs alongside the application services within the same Docker Compose project and shares the microshop-net bridge network.

Service	Host Port	Internal Port	Description
Frontend (Nginx)	80	80	Dashboard - single public entry point
Prometheus	9090	9090	Metrics collection + alert rules
Alertmanager	9093	9093	Alert routing and notification
Loki	3100	3100	Log aggregation backend
Grafana	3000	3000	Dashboards (metrics + logs)
Auth Service	-	8001	JWT authentication (internal only)
Product Service	-	8002	Product catalog (internal only)
Order Service	-	8003	Order management + PostgreSQL
User Service	-	8004	User profiles (internal only)
Chat Service	-	8005	Messaging (internal only)

Microservices on ports 8001–8005 are not bound to the host. They are reachable only through the Nginx reverse proxy or from within the Docker network, which eliminates direct external exposure.

3. Automation and Health Checks

Docker Compose provides the primary automation layer. Each service is configured with restart: unless-stopped so that container crashes trigger automatic restarts without manual intervention. Resource limits (mem_limit, cpus) prevent a single service from starving others.

3.1 Health Checks

Every service declares a Docker health check. The checks use lightweight Python one-liners (urllib.request) rather than curl, which avoids adding curl to images. PostgreSQL uses pg_isready. Services that depend on PostgreSQL (order-service, user-service, chat-service) declare depends_on: condition: service_healthy so they only start once the database is ready.

Service	Health check endpoint	Interval / Retries
postgres	pg_isready -U $DB_USER	10s / 5
auth-service	http://localhost:8001/health	15s / 3
product-service	http://localhost:8002/health	15s / 3
order-service	http://localhost:8003/health	15s / 3
user-service	http://localhost:8004/health	15s / 3
chat-service	http://localhost:8005/health	15s / 3
frontend	wget -qO- http://localhost:80	15s / 3

4. Prometheus Alerting and Alertmanager

Prometheus scrapes all five microservices plus itself and Alertmanager. Alert rules are defined in monitoring/alert_rules.yml and loaded via the rule_files directive. Alerts are forwarded to Alertmanager at alertmanager:9093 via the alerting block in prometheus.yml.

4.1 Alert Rules

Alert Name	Condition	For	Severity
ServiceDown	up == 0	30 s	critical
OrderServiceErrors	increase(order_errors_total[1m]) > 0	30 s	critical
OrderServiceHighLatency	histogram_quantile(0.95, …) > 1	1 m	warning
HighCPUUsage	rate(process_cpu_seconds_total[1m]) * 100 > 80	1 m	warning

4.2 Alertmanager Routing

Alertmanager routes alerts by the severity label. Critical alerts repeat every 15 minutes; warning alerts repeat every hour. An inhibition rule suppresses warning-level alerts for a service when a critical alert for the same service is already firing, reducing noise.

Severity	Receiver	Group wait	Repeat interval
critical	critical-receiver	30 s	15 minutes
warning	default-receiver	30 s	1 hour

5. Incident Simulation and Alert Verification

To verify the full alerting pipeline, the incident from Assignment 4 was reproduced: the DB_HOST environment variable of order-service was set to wrong-host, forcing every database call to fail while the process itself kept running. This confirms the core SRE lesson that process liveness (up == 1) does not equal service health.

5.1 Incident Timeline

Time	Event
T+00:00	DB_HOST changed to wrong-host in docker-compose.yml
T+00:05	order-service restarted: docker compose up -d --force-recreate order-service
T+00:10	Frontend dashboard shows Order Service as Unreachable
T+00:15	/api/orders/orders returns HTTP 503
T+00:20	order_errors_total counter starts incrementing in Prometheus
T+00:25	/health returns 503: DB connection failed: could not translate host name 'wrong-host'
T+00:30	docker logs order-service shows repeated psycopg2 DNS errors
T+00:35	Prometheus fires OrderServiceErrors alert (critical)
T+00:35	Alertmanager receives alert and places it in critical group
T+01:00	Root cause confirmed: wrong DB_HOST value
T+01:05	DB_HOST corrected back to postgres
T+01:10	order-service restarted with correct config
T+01:15	/health returns 200: db: connected
T+01:20	Dashboard shows all services green - incident resolved

```
▷Run Service
order-service:
  build: ./services/order-service
  container_name: order-service
  # No host port binding — reachable
  environment:
    DB_HOST: wrong-host
    DB_PORT: "5432"
    DB_NAME: ordersdb
    DB_USER: ${DB_USER:-postgres}
    DB_PASS: ${DB_PASS:-postgres}
  networks:
    - microshop-net
  depends_on:
    postgres:
```

Figure 15: Simulated incident - DB_HOST set to wrong-host, order-service restarted

5.2 Prometheus Alert Firing

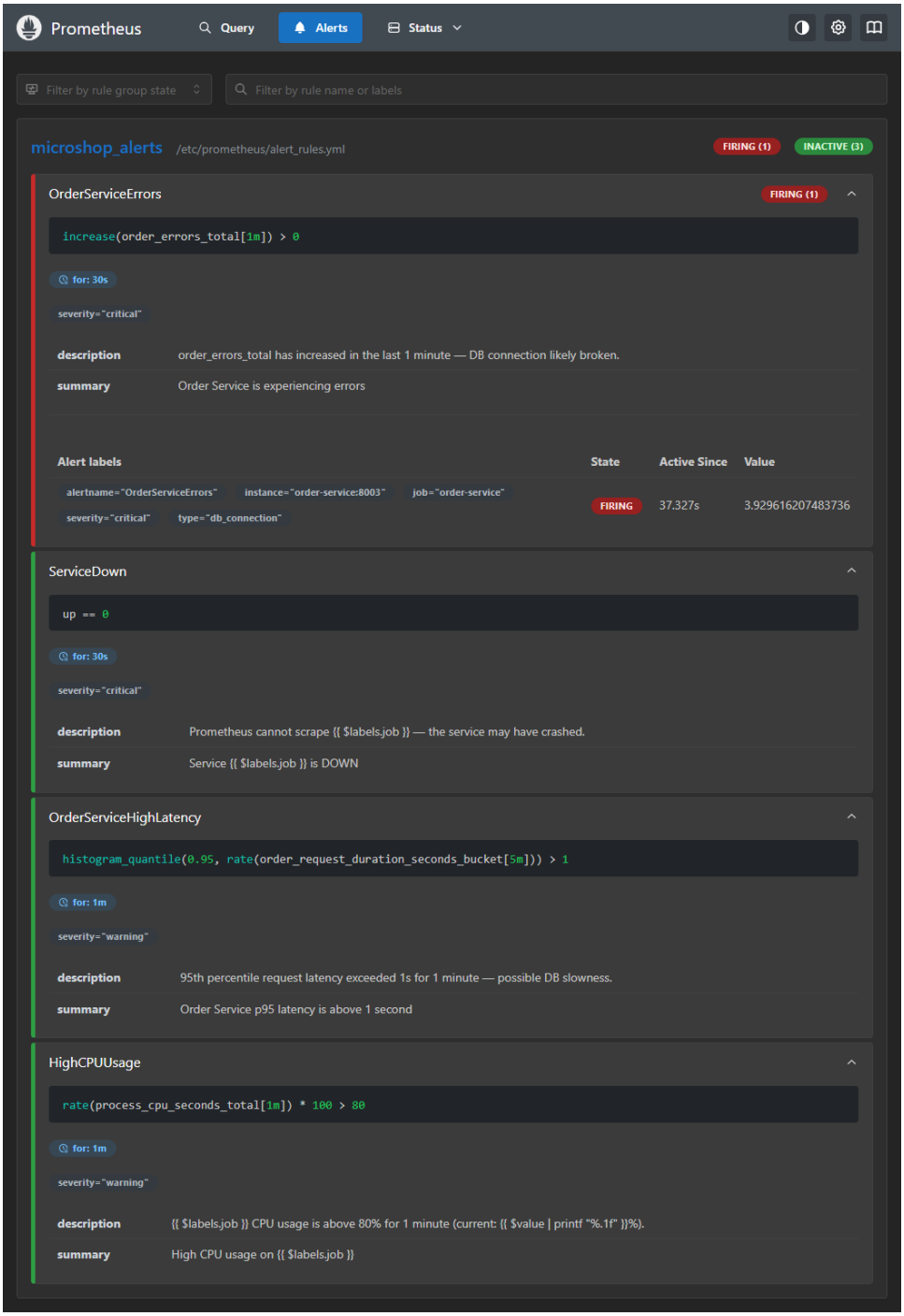

Figure 16: Prometheus Alerts - OrderServiceErrors in FIRING state (critical)

5.3 Alertmanager Receiving the Alert

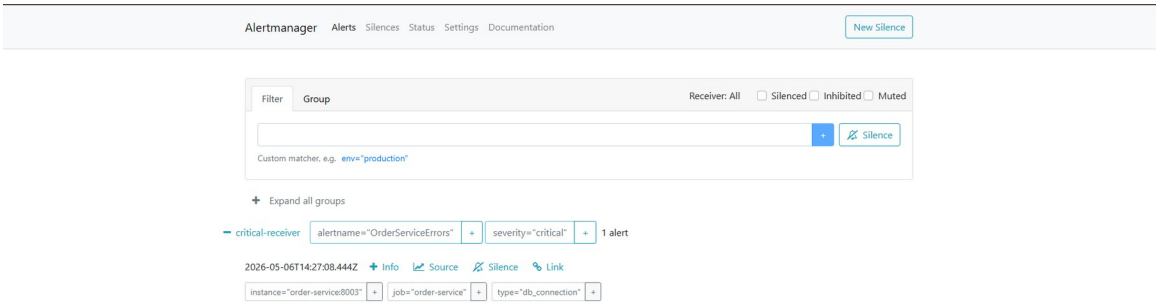

Figure 17: Alertmanager - critical alert group with OrderServiceErrors

5.4 Recovery

After restoring DB_HOST to postgres and restarting order-service, the health endpoint returned 200, the error counter stopped incrementing, and all Prometheus alerts returned to the inactive state. Alertmanager's alert group cleared automatically.

```
▷Run Service
order-service:
  build: ./services/order-service
  container_name: order-service
  # No host port binding — reachable
  environment:
    DB_HOST: postgres
    DB_PORT: "5432"
    DB_NAME: ordersdb
    DB_USER: ${DB_USER:-postgres}
    DB_PASS: ${DB_PASS:-postgres}
  networks:
```

Figure 18: order-service /health returning 200 after fix

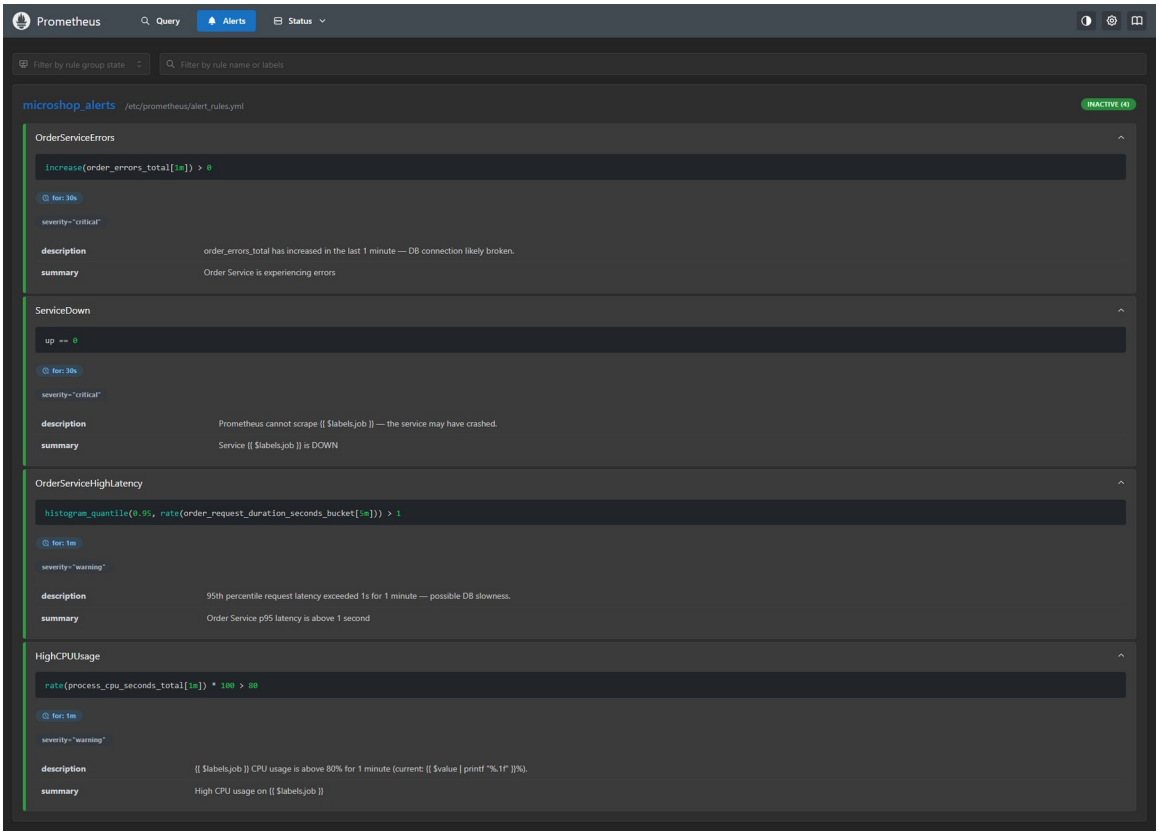

Figure 19: Prometheus after fix - all alerts INACTIVE

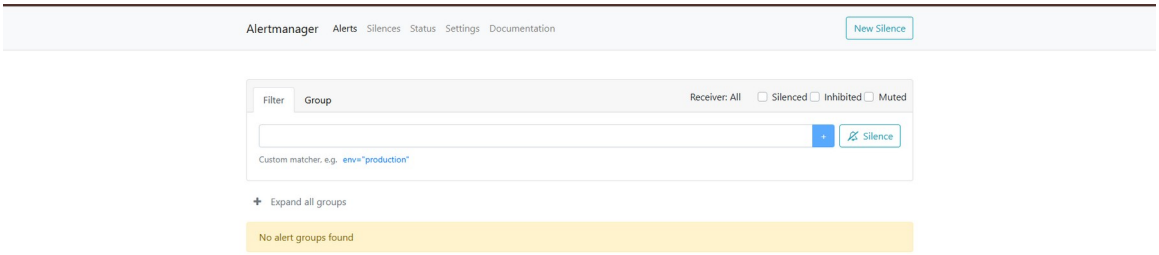

Figure 20: Alertmanager after fix - no active alert groups

6. Centralised Log Aggregation (Loki + Promtail)

Promtail runs as a sidecar container and collects all Docker container logs by reading the Docker socket (/var/run/docker.sock) and the container log directory (/var/lib/docker/containers). Each log stream is tagged with three labels: container, service, and compose_project. Logs are pushed to Loki at http://loki:3100/loki/api/v1/push.

6.1 Loki Configuration

Loki runs in single-node mode with boltdb-shipper as the index store and the local filesystem for chunk storage. The WAL (Write-Ahead Log) is stored at /loki/wal inside the loki-data Docker volume to avoid permission errors. Retention is set to 168 hours (7 days).

6.2 Grafana Loki Integration

The Loki datasource is automatically provisioned via monitoring/grafana/provisioning/datasources/prometheus.yml alongside the Prometheus datasource. The MicroShop dashboard includes a dedicated Logs panel (Panel 9) showing live order-service logs using the query {service="order-service"}.

To explore logs manually in Grafana:

- Open Grafana → Explore
- Select datasource: Loki
- Query: {service="order-service"} - all order-service logs
- Query: {service="order-service"} |= "error" - filter to error lines only

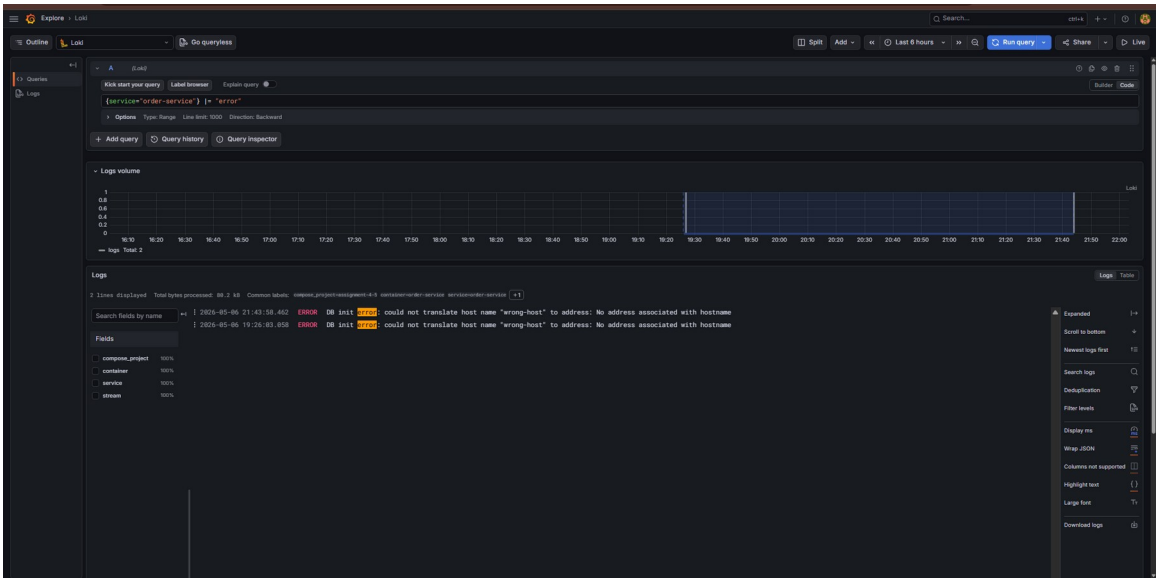

Figure 22: Grafana Explore - Loki query showing order-service error logs during incident

7. Grafana Dashboard

The MicroShop SRE Dashboard is auto-provisioned from monitoring/grafana/provisioning/dashboards/microshop.json. It refreshes every 10 seconds and covers a 15-minute time window by default. For Assignment 6, three new panels were added to the existing dashboard:

Panel	Type	Query / Datasource
Panel 7 - User & Chat Services Request Rate	Time series	Prometheus: rate(user/chat_requests_total[1m])
Panel 8 - Order Service p95 / p50 Latency	Time series	Prometheus: histogram_quantile(0.95/0.50, …)
Panel 9 - Order Service Logs	Logs	Loki: {service="order-service"}

Figure 21: Full Grafana dashboard with all 9 panels including latency and Loki logs

8. Capacity Planning

8.1 Load Simulation

The load_test.py script sends concurrent HTTP requests to all service endpoints through the Nginx reverse proxy and measures requests per second and error rate in real time. The script uses Python threads and argparse for configuration.

Usage:

- python load_test.py (10 workers, 60 seconds)
- python load_test.py --workers 20 --duration 120 (stress test)
- python load_test.py --host http://<EC2_IP> (against AWS instance)

8.2 Metrics Collected

Metric	Prometheus Expression	Visible In
Request rate	rate(order_requests_total[1m])	Grafana dashboard
Error rate	rate(order_errors_total[1m])	Grafana dashboard
p95 latency	histogram_quantile(0.95, rate(order_request_duration_seconds_bucket[5m]))	Grafana dashboard
CPU usage	rate(process_cpu_seconds_total[1m]) * 100	Prometheus / alert
Memory	process_resident_memory_bytes	Prometheus Explore

8.3 Observations Under Load

Under concurrent load the Order Service is the primary bottleneck. Each request opens a new PostgreSQL connection (no connection pooling), so CPU and latency rise sharply as worker count increases. The database connection queue fills up first, causing db_query errors to increment. The other four services (auth, product, user, chat) remain stable under load because their hot paths do not hit the database.

8.4 Scaling Strategies

- Horizontal scaling - run multiple Order Service replicas behind Nginx using deploy.replicas in Docker Swarm or Kubernetes
- Vertical scaling - increase mem_limit and cpus in docker-compose.yml, or upgrade EC2 instance type in terraform.tfvars
- Connection pooling - add PgBouncer in front of PostgreSQL to eliminate per-request connection overhead
- Long-term - migrate to Kubernetes with HPA (Horizontal Pod Autoscaler) triggered on process_cpu_seconds_total

9. Security Hardening

9.1 Docker Compose - Port Isolation

All five microservices (ports 8001–8005) have no host port binding in docker-compose.yml. They are reachable only through the Nginx reverse proxy on port 80 or from within the microshop-net bridge network. This eliminates the risk of direct external access to service APIs.

9.2 Nginx Dynamic DNS Resolution

The Nginx configuration uses Docker's internal DNS resolver (127.0.0.11) with a 10-second TTL and per-request upstream resolution via set $upstream. This prevents startup failures when microservices are not yet ready and ensures that container IP changes (e.g. after restart) are picked up automatically.

9.3 Credentials in .env

Database credentials (DB_USER, DB_PASS), JWT secret (JWT_SECRET), and Grafana admin credentials (GRAFANA_USER, GRAFANA_PASS) are all externalised to the .env file. The .env file is listed in .gitignore. The repository contains only .env.example with placeholder values. The docker-compose.yml references all secrets via ${VAR:-default} expansion so no credentials are hardcoded.

9.4 AWS Security Group

The Terraform security group restricts all non-HTTP ports to allowed_ssh_cidr (the operator's IP/32). Ports 8001–8005 are not opened at all - microservices are never reachable through the AWS firewall. Port 22 (SSH), 3000 (Grafana), 9090 (Prometheus), and 9093 (Alertmanager) are restricted to the allowed CIDR.

Port	Protocol	Source	Purpose
22	TCP	allowed_ssh_cidr	SSH access
80	TCP	0.0.0.0/0	HTTP (Nginx frontend)
3000	TCP	allowed_ssh_cidr	Grafana
9090	TCP	allowed_ssh_cidr	Prometheus
9093	TCP	allowed_ssh_cidr	Alertmanager
8001–8005	-	not opened	Microservices - internal only

10. Postmortem

Incident: Order Service DB Connection Failure

Field	Details
Duration	~20 minutes (simulated)
Impact	All order endpoints returned 503; order_errors_total rose monotonically
Root cause	DB_HOST misconfigured to 'wrong-host' - DNS resolution failed on every request
Detection	Prometheus OrderServiceErrors alert fired within 35 seconds; Grafana dashboard showed error rate spike
Resolution	DB_HOST corrected to 'postgres'; order-service restarted
Key lesson	process liveness (up == 1) does not equal service health - always instrument /health endpoints

Action Items

- DONE - OrderServiceErrors alert rule added to alert_rules.yml
- DONE - ServiceDown alert rule added (up == 0, critical, 30 s)
- DONE - OrderServiceHighLatency alert added (p95 > 1 s, warning, 1 m)
- DONE - HighCPUUsage alert added (CPU > 80%, warning, 1 m)
- DONE - Alertmanager routing by severity (critical 15 min / warning 1 h)
- DONE - Loki + Promtail deployed and log pipeline verified
- DONE - Grafana Loki datasource and logs panel auto-provisioned
- DONE - Load simulation script (load_test.py) added
- DONE - Microservice host port bindings removed (Docker security)
- DONE - AWS Security Group: Alertmanager port 9093 restricted; 8001–8005 not opened
- DONE - DB credentials moved to .env; .env.example added to repo
- TODO - Replace webhook placeholder (localhost:5001) with real Slack/PagerDuty endpoint
- TODO - Add PgBouncer connection pooler in front of PostgreSQL
- TODO - Migrate to Kubernetes with HPA for production scaling

11. Deployment Guide

Local (Docker Compose)

- cp .env.example .env - then edit .env to set real credentials
- docker compose up --build - builds and starts all 12 services
- Open http://localhost:80 for the dashboard
- Open http://localhost:9090 for Prometheus
- Open http://localhost:9093 for Alertmanager
- Open http://localhost:3000 for Grafana (credentials from .env)

AWS EC2 (Terraform)

- cd terraform
- curl -s ifconfig.me - note your public IP
- Set allowed_ssh_cidr = "YOUR_IP/32" in terraform.tfvars
- aws configure - set access key, secret, region
- terraform init && terraform plan && terraform apply
- terraform destroy - tear down to avoid charges

Load Testing

- python load_test.py - default: 10 workers, 60 seconds, localhost
- python load_test.py --workers 20 --duration 120 - stress test
- python load_test.py --host http://<EC2_IP> - against AWS instance